Auditoría Integral de Calidad de Software en la 4RI: PatentGuard CO

Informe Final de Auditoría Técnica, Seguridad, Madurez CMMI y Análisis Crítico

Samuel Mendoza Loaiza y Andrés Felipe Gualteros Tovar

Institución Universitaria de Envigado

Facultad de Ingeniería — Asignatura: Calidad de Software

10 de noviembre de 2026

Resumen Ejecutivo

PatentGuard CO es una API REST académica que consulta registros públicos de patentes, aplica validaciones deterministas de integridad y, cuando está configurado, incorpora una evaluación cualitativa mediante Gemini. Su propósito es apoyar la revisión técnica de disputas de autoría sin reemplazar una decisión humana, administrativa o judicial.

La auditoría consolidó la evidencia del Trabajo 1 y las mejoras posteriores de calidad. Se revisaron el código de aplicación, esquemas Pydantic, pruebas, configuración de secretos, documentación y resultados de pruebas automatizadas. El resultado es favorable para un entorno académico y de demostración: la solución está modularizada, controla fallos previsibles de sus proveedores, separa hechos verificables de la valoración de IA y cuenta con pruebas reproducibles. La última evidencia disponible registra **24 pruebas aprobadas**.

> **Dictamen de Madurez y Exposición:** El sistema no debe exponerse públicamente como servicio productivo sin una fase adicional de fortalecimiento. Los principales riesgos son la falta de autenticación y limitación de tasa, la ausencia de límite en el número de solicitudes por petición, llamadas externas secuenciales, la posible transferencia de datos personales a Gemini y la falta de controles operativos de despliegue. El diagnóstico de madurez es **Nivel 2 — Gestionado**, como estimación académica y no como certificación formal CMMI.

1. Objeto, Alcance y Método de Auditoría

1.1 Objeto Auditado

Se auditó PatentGuard CO, la API del Trabajo 1 y sus incrementos de calidad. El flujo principal recibe números de solicitud y una descripción de conflicto; consulta el portal de datos abiertos, normaliza los registros, ejecuta reglas de integridad y, opcionalmente, solicita a Gemini un análisis cualitativo estructurado.

1.2 Alcance

La revisión comprendió los siguientes artefactos del repositorio:

- Código de aplicación: rutas, modelos, cliente de datos abiertos, validación, configuración y servicio Gemini.
- Pruebas unitarias, de API, seguridad y rendimiento en memoria.
- Configuración de secretos mediante `.gitignore` y plantilla `.env.example`.
- Documentación técnica, de pruebas, uso responsable de IA y auditorías de Fase 3.
- Evidencia de ejecución conservada en `docs/evidencia_ejecucion_pruebas.txt`.

No se realizaron pruebas de penetración, análisis dinámico contra proveedores reales, pruebas de carga de infraestructura ni una evaluación oficial CMMI. Las conclusiones de seguridad y madurez son técnicas y académicas, basadas en la evidencia disponible.

1.3 Criterios y Método

Se aplicó revisión estática, lectura de trazabilidad requisito–módulo–prueba, inspección de controles de configuración y ejecución local de la suite. Los criterios evaluados fueron: corrección funcional, robustez, seguridad de entrada y secretos, privacidad, mantenibilidad, eficiencia, documentación y gestión del proceso.

2. Descripción Técnica del Sistema

Componente	Responsabilidad	Valor de Calidad
FastAPI (`main.py`)	Publica `/` , `/health` y `POST /analizar-conflicto` ; coordina el flujo.	Contrato REST y respuesta estructurada.
Pydantic (`schemas.py`)	Valida carga de entrada y modela respuesta.	Rechaza cargas inválidas con HTTP 422 y limita longitudes de campos.
Cliente Socrata (`client_datos_abiertos.py`)	Consulta un recurso de datos abiertos con filtro y tiempo máximo.	Evita descargar el conjunto completo y controla fallos HTTP/red/JSON.
Validador (`validator.py`)	Revisa campos, tipos, fechas, años y duplicados.	Separa errores, advertencias y anomalías reproducibles.
Servicio Gemini (`gemini_service.py`)	Solicita un análisis cualitativo JSON con limitaciones éticas.	Complementa, sin sustituir, las reglas deterministas.
Pruebas (`tests/`)	Simulan dependencias externas y prueban rutas y reglas.	Evidencia repetible sin consumir proveedores reales.

La arquitectura distingue entre evidencia técnica y conclusión cualitativa. La ausencia de inventor o una anomalía de fecha genera una señal revisable, pero no prueba plagio, fraude ni titularidad. Esa distinción es esencial en un sistema que trata disputas reales o hipotéticas.

3. Resultados Verificables de Pruebas

La evidencia almacenada en `docs/evidencia_ejecucion_pruebas.txt` reporta una ejecución con Python 3.13.1 y pytest 9.1.1: **24 pruebas aprobadas en 1,34 segundos**, sin llamadas de red ni consumo de Gemini. La auditoría también verificó localmente el estado actual: **24 pruebas aprobadas**, con una advertencia de deprecación proveniente de `fastapi.testclient` /Starlette, no de la lógica de negocio.

Área de Prueba	Evidencia	Resultado y Alcance
API	Salud del servicio, flujo exitoso y carga inválida.	Rutas responden JSON, el flujo simulado integra consulta, validación y Gemini, y Pydantic responde 422 ante errores.
Datos abiertos	Resultado encontrado/no encontrado, HTTP, timeout y JSON inválido.	El cliente transforma fallos externos en excepción controlada.
Validador	Registros válidos, obligatorios ausentes, recomendados ausentes, tipos, fechas, rango de año y duplicados.	Las reglas son deterministas y se ejecutan sin IA.

Área de Prueba	Evidencia	Resultado y Alcance
Gemini	Éxito simulado, clave ausente, cuota agotada y JSON inválido.	El servicio controla escenarios previsibles del proveedor.
Seguridad	Intento de inyección SoQL, no filtración de secretos/rutas y cargas excesivas o corruptas.	Se confirma limpieza básica, ocultación de detalles en respuesta y límites por campo.
Rendimiento / estabilidad	25 solicitudes concurrentes en memoria con dependencias simuladas.	Se confirma aislamiento observable y promedio interno menor a 150 ms en el escenario simulado.

La prueba de concurrencia no representa una garantía de rendimiento de producción: no mide latencia de la red, cuota de Gemini, saturación de procesos ni capacidad de infraestructura. Su valor es demostrar que el endpoint mantiene el comportamiento esperado con servicios simulados y solicitudes concurrentes en memoria.

4. Consolidado de Auditoría Técnica

4.1 Calidad del Producto y Contrato REST

La interfaz REST resulta adecuada para el alcance académico. `GET /health` permite una comprobación básica, mientras `POST /analizar-conflicto` retorna registros encontrados, solicitudes no encontradas, inconsistencias, análisis ético y limitaciones. Esta estructura evita ocultar resultados parciales y favorece una interpretación responsable.

Pydantic establece un mínimo de una solicitud, descripción entre 10 y 5.000 caracteres, solicitud de hasta 100 caracteres e inventor reclamante de hasta 200. Son controles efectivos contra varias entradas malformadas. Como brecha, no hay máximo para la cantidad de solicitudes ni de inventores; una carga con miles de elementos puede degradar el servicio incluso si cada texto individual cumple su límite.

La respuesta usa diccionarios flexibles para los registros y métricas. Para producción conviene un modelo de salida explícito, con campos permitidos, y un estado tipado mediante `Enum` o `Literal`; así se reduce el riesgo de que una fuente externa introduzca columnas inesperadas o rompa clientes consumidores.

4.2 Integridad, Complejidad y Eficiencia

La validación de un registro es O(1) en tiempo y memoria adicional: realiza un número fijo de comprobaciones y hasta dos conversiones de fecha. La validación de un conjunto de n registros es O(n) en tiempo y O(n) en memoria, gracias al uso de conjuntos para detectar solicitudes duplicadas. Es una elección eficiente para lotes moderados.

El cliente de datos abiertos usa `httpx` asíncrono, un límite de 10 segundos y una consulta con filtro y una sola coincidencia. Sin embargo, el endpoint recorre las solicitudes secuencialmente y crea un cliente HTTP por consulta; en el peor caso, la latencia se acumula. Además, la llamada a Gemini es síncrona dentro de una ruta `async`, por lo que puede bloquear un *worker* mientras espera la red. Estas decisiones son aceptables para un prototipo de bajo volumen, pero no para una carga pública sostenida.

4.3 Estabilidad y Despliegue

Riesgo Operativo	Control Existente	Brecha y Acción Recomendada
Falla de datos abiertos	Tiempo máximo, excepción propia y respuesta parcial.	Añadir reintentos acotados, métricas y alertas.
Cuota o caída de Gemini	Mapeo de errores frecuentes a limitación controlada.	Aplicar presupuesto, cuota por usuario, tiempo máximo y modo degradado.
Carga abusiva	Límites de texto y validación de tipos.	Limitar cardinalidad, tamaño de cuerpo y tasa de solicitudes.
Configuración	Dependencias fijadas, plantilla sin secreto y exclusión de `.env`.	Usar gestor de secretos, validar configuración al inicio y alinear variables declaradas con su uso real.
Observabilidad	Registros básicos de errores.	Implementar logs estructurados, correlación, métricas y redacción de datos sensibles.
Liberación	Pruebas locales reproducibles.	Crear CI, análisis de dependencias, escaneo de secretos y entorno de pruebas controlado.

5. Seguridad e Ingeniería Inversa

5.1 Controles Verificados

La configuración ignora archivos de entorno, llaves, certificados y directorios de secretos. La plantilla de entorno contiene marcadores, no una credencial real. La API no devuelve al cliente el detalle de una excepción de Gemini o de datos abiertos, lo que reduce la exposición de rutas internas y mensajes con contexto sensible. También se elimina la comilla simple y los caracteres de control antes de construir el filtro SoQL; una prueba verifica este comportamiento. No se identificaron secretos, claves reales ni rutas locales sensibles en este informe ni en el contenido rastreado revisado.

5.2 Riesgos Críticos

1. **Endpoint público sin autenticación ni limitación de tasa:** Si se despliega tal como está, un tercero puede usar el análisis, agotar cuota de Gemini y aumentar consumo de los proveedores.
2. **Lote no acotado:** La ausencia de máximo de elementos en la lista permite solicitudes muy costosas; al procesarse secuencialmente, su latencia puede crecer linealmente.
3. **Gestión de secreto solo por entorno:** Es correcta para desarrollo, pero faltan rotación, mínimo privilegio, auditoría de uso y procedimiento de revocación.

4. **Privacidad y reputación:** Los nombres de inventores y descripciones de conflictos pueden ser datos personales o contextualizar acusaciones no verificadas. Su publicación o envío a un servicio de IA requiere minimización, información al usuario y retención definida.
5. **Contrato de datos externo:** El uso de estructuras flexibles y `raw_data` puede propagar campos no previstos desde la fuente pública.

5.3 Estimación Defensiva de Ingeniería Inversa

La seguridad no puede basarse en ocultar el código o los endpoints. Si la API se expone sin autenticación y conserva documentación OpenAPI, una persona con conocimientos técnicos podría identificar sus rutas, esquema y respuestas en **15 a 60 minutos**. Sin documentación, el descubrimiento de la superficie básica puede tomar **2 a 8 horas** mediante pruebas moderadas. Si se distribuye un contenedor o bytecode Python, la reconstrucción aproximada de módulos y reglas puede tomar **1 a 2 días**. Una comprensión operativa más amplia, incluyendo proveedores y límites, puede requerir **3 a 10 días** según el acceso disponible.

Estas estimaciones son defensivas y no autorizan pruebas sobre sistemas ajenos. El control efectivo es autenticación, autorización, reducción de privilegios, límites de tasa, monitoreo y protección de secretos; la ofuscación solo añade fricción limitada.

6. Marco de Ingeniería de Software: Pressman y SCRUM

El proyecto refleja un ciclo iterativo consistente con las actividades de marco de Roger Pressman: comunicación, planificación, modelado, construcción y retroalimentación. En el Trabajo 1 se construyó el flujo funcional: API, fuente abierta, validación e IA cualitativa. En el Trabajo 2 se tomó retroalimentación técnica para endurecer entradas, fijar dependencias, reducir exposición de errores y añadir pruebas de seguridad y concurrencia. La auditoría final transforma esos resultados en riesgos, evidencia y acciones de mejora.

Actividad Pressman	Aplicación en PatentGuard CO
Comunicación	Delimitación del conflicto ético entre autoría y lucro, fuentes y limitaciones.
Planificación	División en módulos y fases: funcionalidad, endurecimiento, pruebas no funcionales y auditoría.
Modelado	Esquemas Pydantic, reglas de integridad y contrato de respuesta.
Construcción	FastAPI, cliente Socrata, validador y servicio Gemini; correcciones de Fase 1 y 2.
Retroalimentación	Resultados de pytest, documentación de evidencia, hallazgos de seguridad y plan de remediación.

SCRUM se aprecia como adaptación ligera, no como implementación formal: el backlog se organizó en fases con incrementos verificables. Para madurar esta práctica se recomienda definir objetivo por sprint, criterios de aceptación, responsable, riesgo, revisión y retrospectiva. Una definición de terminado debe incluir revisión de código, pruebas verdes, documentación actualizada y análisis proporcional de seguridad y privacidad.

7. Evaluación CMMI

7.1 Diagnóstico: Nivel 2 — Gestionado

El proyecto presenta evidencia compatible con un **Nivel 2 — Gestionado** a escala académica: requisitos documentados, fases reconocibles, control de versiones, pruebas repetibles, dependencias fijadas y resultados de verificación. No constituye una valoración oficial CMMI, pues no hay evidencia de una evaluación institucional, aplicación sostenida en varios proyectos ni todos los artefactos de gestión organizacional requeridos.

Fortalezas	Debilidades que Impiden Aproximarse al Nivel 3
Requisitos y arquitectura documentados.	No existe backlog formal con criterio de aceptación, prioridad, cambios aprobados y responsables.
Pruebas automatizadas de unidad, API, seguridad y concurrencia.	No hay CI, umbrales de calidad, cobertura ni revisión independiente sistemática.
Git, rama de trabajo, dependencias fijas y exclusión de secretos.	No se observa estrategia de *releases*, baseline, SBOM o configuración de despliegue reproducible.
Riesgos técnicos identificados y mitigaciones puntuales.	Falta registro de riesgos con probabilidad, impacto, propietario y contingencia.
Documentación amplia y trazabilidad básica.	No existe proceso estándar institucional reutilizable ni medición histórica.

7.2 Ruta de Mejora hacia Nivel 3 — Definido

1. Definir y versionar un proceso mínimo de requisitos, diseño, desarrollo, revisión, pruebas, liberación e incidentes.
2. Usar plantillas para historias, criterios de aceptación, casos de prueba, riesgos, decisiones de arquitectura y revisiones de cambios.
3. Mantener trazabilidad viva entre requisito, módulo, prueba y evidencia; exigirla en cada cambio.
4. Automatizar CI: pruebas, formato/lint, análisis de dependencias y escaneo de secretos.
5. Medir tendencias de defectos, cobertura, latencia, fallos de proveedores e incidentes de seguridad.
6. Establecer controles de privacidad, secretos, retención de logs y respuesta a incidentes.

El objetivo del Nivel 3 no es añadir burocracia, sino hacer que las buenas prácticas ya observadas se ejecuten de manera definida, repetible y sostenible por cualquier integrante del equipo.

8. Análisis Crítico: Respuestas a las Preguntas Obligatorias

8.1 ¿El sistema auditado es confiable?

Es confiable de forma condicionada para un entorno académico o de demostración controlada; no es todavía confiable como servicio público de decisión o evidencia legal. La evidencia favorable incluye 24 pruebas aprobadas, validación determinista de campos y fechas, manejo de fallos HTTP/timeout/JSON, respuestas parciales controladas y separación explícita entre hechos y análisis de IA. La confiabilidad está limitada porque no se probaron proveedores reales de forma automatizada, no existen métricas operativas ni CI, y faltan controles contra abuso y fallas de producción. Además, el propio sistema declara correctamente que no emite un dictamen jurídico vinculante.

8.2 ¿Qué riesgos críticos existen?

Los riesgos críticos son: exposición sin autenticación ni límite de tasa; consumo ilimitado por lista de solicitudes no acotada; bloqueo o degradación por llamadas externas secuenciales y llamada Gemini síncrona; filtración o abuso de la credencial de Gemini si se gestiona fuera del repositorio de forma insegura; y tratamiento de nombres/descripciones de conflicto sin una política explícita de privacidad y minimización. También hay riesgo de contrato inestable por usar datos externos flexibles y propagar información sin una lista de campos permitidos.

8.3 ¿Qué nivel de madurez presenta?

El diagnóstico fundamentado es **Nivel 2 — Gestionado**, en contexto académico. Se observan fases, requisitos, pruebas, evidencia, control de versiones y correcciones a partir de hallazgos. No se puede afirmar certificación formal: faltan procesos organizacionales definidos y repetibles, indicadores históricos, gestión de riesgos formal, CI y evidencia de aplicación transversal. El objetivo siguiente es Nivel 3 — Definido mediante procesos estándar, trazabilidad viva y mejora basada en métricas.

8.4 ¿Qué mejoras recomendarían?

Prioridad inmediata: limitar la cantidad de solicitudes e inventores, autenticar la ruta de análisis y aplicar límite de tasa; validar localmente la respuesta Gemini contra el esquema; tipar y filtrar la salida de datos; reutilizar clientes HTTP, aplicar concurrencia acotada y aislar la llamada bloqueante de Gemini. A corto plazo: gestor de secretos, rotación, CI, análisis de dependencias, escaneo de secretos, métricas, logs con redacción de PII y entorno de pruebas. A nivel de proceso: política de privacidad/retención, registro de riesgos, ADR y matriz de trazabilidad mantenida en cada incremento.

8.5 ¿Qué impacto tendría una falla en la vida real?

Una respuesta equivocada o presentada sin sus limitaciones podría afectar reputaciones, relaciones laborales o decisiones de una disputa de autoría. Una indisponibilidad impediría revisar información en el momento necesario; el abuso de la API podría agotar cuota o generar costos; la exposición de una credencial permitiría uso no autorizado; y la publicación de nombres junto a alegaciones no verificadas podría causar daño de privacidad y reputación. Por eso la API debe apoyar evidencia y revisión humana, no reemplazarla, y debe desplegarse con controles de seguridad y privacidad antes de manejar casos reales.

8.6 ¿Qué defectos encontraron en la API del Trabajo 1 y cuáles lograron corregir o mejorar mediante las pruebas y la auditoría?

Defecto o Debilidad Inicial	Corrección / Mejora Aplicada	Evidencia
Uso de opciones Pydantic deprecadas para mínimos y ejemplos.	Actualización a `min_length` y ejemplos mediante `json_schema_extra`.	Esquemas actualizados y documentación de Fase 1.
Dependencias no alineadas de forma precisa con el SDK utilizado.	Versiones fijadas y uso de `google-genai`.	`requirements.txt` y suite de Gemini.
La consulta SoQL incorporaba directamente el identificador en el filtro.	Eliminación de comillas simples y caracteres de control antes de construir el filtro.	Prueba de sanitización de seguridad.
Riesgo de reflejar detalles de error del proveedor.	La ruta expone mensajes genéricos al consumidor y conserva la limitación funcional.	Prueba de no filtración de secreto/ruta.
No se exponían explícitamente solicitudes no encontradas ni tiempo de proceso.	Se añadieron ambas métricas a la respuesta.	Prueba de API y modelo de respuesta.
No existían pruebas no funcionales.	Se agregaron pruebas de cargas corruptas, inyección básica, confidencialidad y 25 solicitudes concurrentes simuladas.	Evidencia de 24 pruebas aprobadas.

La auditoría también detectó defectos pendientes de corrección: sin autenticación, sin límite de tasa, sin máximo de elementos por carga, configuración parcialmente no utilizada, salida flexible de datos y posible bloqueo por llamada Gemini síncrona. Estos hallazgos alimentan el siguiente backlog y no deben ocultarse bajo el resultado positivo de la suite.

9. Conclusiones y Plan Priorizado

PatentGuard CO cumple el objetivo académico de demostrar una API con datos abiertos, validación de calidad y una IA usada de forma auxiliar y no concluyente. La combinación de pruebas deterministas, manejo de fallos y documentación le da una base confiable para aprendizaje y demostración. La madurez alcanzada no elimina los riesgos de una exposición real.

Prioridad	Acción	Resultado Esperado
Alta	Autenticación, autorización y límite de tasa.	Reducir abuso, consumo indebido y exposición de la API.
Alta	Límite de cardinalidad y tamaño total de carga.	Evitar degradación por peticiones costosas.
Alta	Gestión profesional de secretos y procedimiento de rotación.	Reducir impacto de fuga de credenciales.
Media	Modelos de salida tipados, filtrado de datos y validación local de respuesta IA.	Contrato estable y menor exposición de datos.
Media	Cliente HTTP reutilizable, concurrencia acotada y aislamiento de Gemini bloqueante.	Mejor latencia y estabilidad bajo carga.
Media	Política de privacidad, minimización, retención y revisión humana.	Tratamiento responsable de inventores y conflictos.
Media	CI, métricas y escaneo de dependencias/secretos.	Calidad repetible y detección temprana de regresiones.

Referencias

CMMI Institute. (s. f.). *CMMI: Strengthening capabilities for high performance*. Recuperado el 10 de noviembre de 2026, de https://cmmiinstitute.com/

FastAPI. (s. f.). *FastAPI documentation*. Recuperado el 10 de noviembre de 2026, de https://fastapi.tiangolo.com/

Google. (s. f.). *Gemini API documentation*. Recuperado el 10 de noviembre de 2026, de https://ai.google.dev/gemini-api/docs

Krekel, H., Oliveira, B., Pfannschmidt, R., Bruynooghe, F., Laugher, B., & Bruhin, F. (s. f.). *pytest documentation*. Recuperado el 10 de noviembre de 2026, de https://docs.pytest.org/en/stable/

Pressman, R. S., & Maxim, B. R. (2019). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Socrata. (s. f.). *Socrata Open Data API developer documentation*. Recuperado el 10 de noviembre de 2026, de https://dev.socrata.com/

Anexo A. Declaración de Integridad del Informe

Este informe se elaboró a partir de código, documentación y resultados de pruebas disponibles en el repositorio. No contiene credenciales, valores de claves, tokens ni rutas locales sensibles. Las estimaciones de ingeniería inversa son defensivas, se presentan para priorizar controles y no constituyen instrucciones de intrusión.